

ESE 5105: Embedded Systems

Introduction to ARM Cortex processor, Variants of Cortex and ARM versions, Comparison of M-series processor, Architecture, Programmers Model, APSR register, Memory Model, Exception Interrupts, Reset

BY:

RAGHUDATHESH G P

Assistant Professor

Manipal School of Information Sciences (MSIS),

MIT Campus, MAHE, Manipal – 576104

Mail: raghudathesh.gp@manipal.edu

Website: raghudathesh.weebly.com

Quotes:

- Great relationships aren't built in a day - great relationships are built daily.
- Train your mind to be stronger than your emotion or else you will loose yourself every time.
- Be valuable not available.
- Kindness is a beauty that never fades.
- Do good, it will come back to you in unexpected ways.
- A person who chases two rabbits catches neither. stay focused on a goal.

The Cortex-M3 and Cortex-M4 Processors:

- The Cortex-M3 and Cortex-M4 are processors designed by ARM.
- The Cortex-M3 processor was the first of the Cortex generation of processors, released by ARM in 2005.
- The Cortex-M4 processor was released in 2010.
- The Cortex-M3 and Cortex-M4 processors use a 32-bit architecture. Internal registers in the register bank, the data path, and the bus interfaces are all 32 bits wide.
- The Instruction Set Architecture (ISA) in the Cortex-M processors is called the Thumb ISA and is based on Thumb-2 Technology which supports a mixture of 16-bit and 32-bit instructions.
- The Cortex-M3 and Cortex-M4 processors have:
 - Three-stage pipeline design
 - Harvard bus architecture with unified memory space: instructions and data use the same address space
 - 32-bit addressing, supporting 4GB of memory space
 - On-chip bus interfaces based on ARM AMBA (Advanced Microcontroller Bus Architecture) Technology, which allow pipelined bus operations for higher throughput
 - An interrupt controller called NVIC (Nested Vectored Interrupt Controller) supporting up to **240 interrupt requests** and from **8 to 256 interrupt priority levels** (dependent on the actual device implementation)
 - Support for various features for Operating System(OS) implementation such as a system tick timer, shadowed stack pointer
 - Sleep mode support and various low power features
 - Support for an optional Memory Protection Unit(MPU) to provide memory protection features like programmable memory, or access permission control
 - Support for bit-data accesses in two specific memory regions using a feature called Bit Band
 - The option of being used in single processor or multi-processor designs
- The ISA used in Cortex-M3 and Cortex-M4 processors provides a wide range of instructions:
 - General data processing, including hardware divide instructions

- Memory access instructions supporting 8-bit, 16-bit, 32-bit, and 64-bit data, as well as instructions for transferring multiple 32-bit data
 - Instructions for bit field processing
 - Multiply Accumulate (MAC) and saturate instructions
 - Instructions for branches, conditional branches and function calls
 - Instructions for system control, OS support, etc.
- Both the Cortex-M3 and Cortex-M4 processors are widely used in modern microcontroller products, as well as other specialized silicon designs such as System on Chips (SoC) and Application Specific Standard Products (ASSP).
 - In general, the ARM Cortex-M processors are regarded as RISC (Reduced Instruction Set Computing) processors.
 - Some might argue that certain characteristics of the Cortex-M3 and Cortex-M4 processors, such as the rich instruction set and mixed instruction sizes, are closer to CISC (Complex Instruction Set Computing) processors.
 - But as processor technologies advance, the instruction sets of most RISC processors are also getting more complex, so much so that this traditional boundary between RISC and CISC processor definition can no longer be applied.

The Cortex-M Processor Family:

- The Cortex-M3 and Cortex-M4 processors are two of the products in the ARM Cortex-M processor family. The whole Cortex-M processor family is shown in figure below.

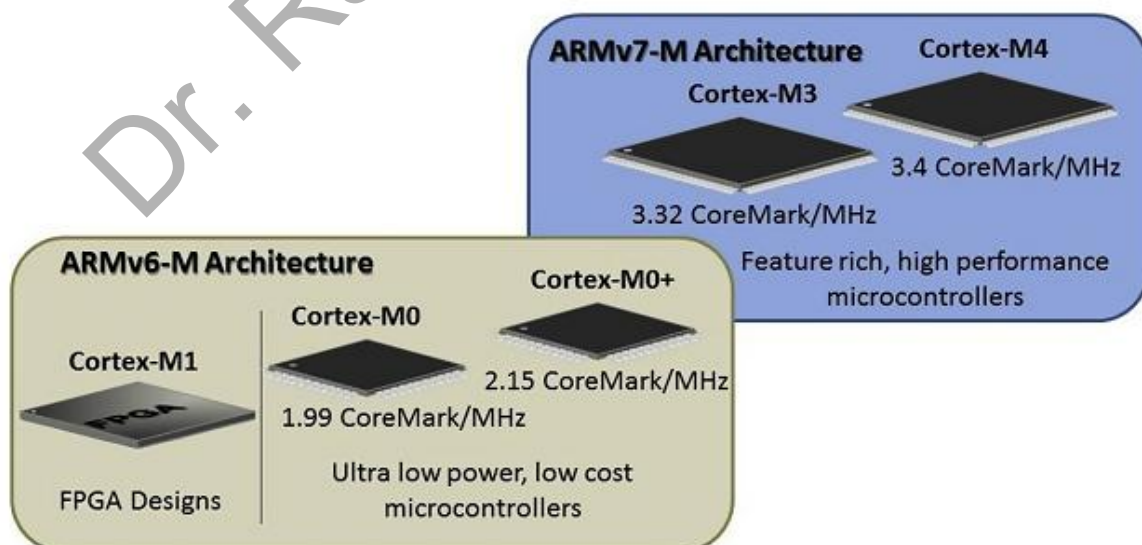


Figure: The Cortex-M processor family

- The Cortex-M3 and Cortex-M4 processors are based on ARMv7-M architecture. Both are high-performance processors that are designed for microcontrollers.
- Because the Cortex-M4 processor has Single Instruction Multiple Data (SIMD), fast Multiply Accumulate (MAC), and saturate arithmetic instructions, it can also carry out some of the digital signal processing applications that traditionally have been carried out by a separate Digital Signal Processor (DSP).
- The Cortex-M0, Cortex-M0+, and the Cortex-M1 processors are based on ARMv6-M, which has a smaller instruction set.
- Both Cortex-M0 and Cortex-M0+ are very small size in terms of gate count, with just about 12K gates in minimum configuration, and are ideal for low-cost microcontroller products.
- The Cortex-M0+ processor has the most state-of-the-art low power optimizations, and has more available optional features.
- The Cortex-M1 processor is designed specifically for FPGA applications. It has Tightly Coupled Memory (TCM) features that can be implemented using memories inside the FPGA, and the design allows high clock frequency operations in advanced FPGA.
- For **general data processing and I/O control tasks**, the Cortex-M0 and Cortex-M0+ processors have excellent energy efficiency due to the low gate count design.
- But for applications with **complex data processing requirements**, they may take **more instructions and clock cycles**.
- In above case, the Cortex-M3 or Cortex-M4 processor would be more suitable, because the additional instructions available in these processors allow the processing to be carried out with fewer instructions compared to ARMv6-M architecture. As a result, we need different processors for different applications.

Advantages of the Cortex-M Processors:

1. Low Power:

- Compared to other 32-bit processor designs, Cortex -M processors are relatively small. The Cortex-M processor designs are also optimized for low power consumption.
- Currently, many Cortex-M microcontrollers have power consumption of less than 200 mA/MHz, with some of them well under 100 mA/MHz.
- the Cortex-M processors also include support for sleep mode features and can be used with various advanced ultra-low power design technologies.

2. Code Density:

- The Thumb ISA provides excellent code density results in reduce cost and power consumption by using a microcontroller with smaller flash memory size, and chip manufacturers can produce microcontroller chips with smaller packages.

3. Interrupts:

- The Cortex-M3 and Cortex-M4 processors have a configurable interrupt controller design, which can support up to **240 vectored interrupts** and multiple levels of interrupt priorities (**from 8 to 256 levels**).
- Nesting of interrupts is automatically handled by hardware, and the interrupt latency is only 12 clock cycles for systems with zero wait state memory.
- The interrupt processing capability makes the Cortex-M processors suitable for many **real-time control applications**.

4. Ease of use, C friendly:

- The Cortex-M processors are very easy to use. In fact, they are easier than compared to many 8-bit processors because Cortex-M processors have a **simple, linear memory map**, and there are **no special architectural restrictions**, which you often find in 8-bit microcontrollers (e.g., memory banking, limited stack levels, etc.).
- You can program almost everything in C including the interrupt handlers.

5. Scalability:

- The Cortex -M processor family allows easy scaling of designs from low-cost, simple microcontrollers costing less than a dollar to high-end microcontrollers running at 200MHz or more.

6. Debug Features:

- With standard design features, which you can find in most microcontrollers like halting and single stepping, you can also generate a **trace to capture program flow, data changes, profiling information**, and so on.

7. OS Support:

- The Cortex-M processors are designed with OS applications in mind. A number of features are available to make OS implementation easier and make OS operations more efficient.
- Currently there are over **30 embedded OSs available for Cortex-M processors**.

8. Versatile System Features:

- The Cortex-M3 and Cortex-M4 processors support a number of system features such as bit addressable memory range (bit band feature) and Memory Protection Unit(MPU).

9. Software Portability and Reusability:

- Since the architecture is very C friendly, you can program almost everything in standard ANSI C.
- One of ARM's initiatives called CMSIS (Cortex Microcontroller Software Interface Standard) makes programming for Cortex-M processor based products even easier by providing standard header files and an API for standard Cortex-M processor functions.
- This allows better software reusability and also makes porting application code easier.

10. Choices (devices, tools, OS, etc.):

- A wide range of choices for software development/debug tools, embedded OS, middleware, etc.

Introduction to the Architecture:

- The Cortex-M3 and Cortex-M4 Processors are based on the ARMv7-M architecture.
- The original ARMv7-M architecture was defined when the Cortex-M3 processor was developed, and when the Cortex-M4 was released, the architecture was extended to include additional instructions and architectural features.
- The extended architecture is sometimes called ARMv7E-M architecture.

Programmer's Model:

Operation Modes and States:

- The Cortex-M3 and Cortex-M4 processors have two operation states and two modes. In addition, the processors can have privileged and unprivileged access levels. These are shown in Figure 1.

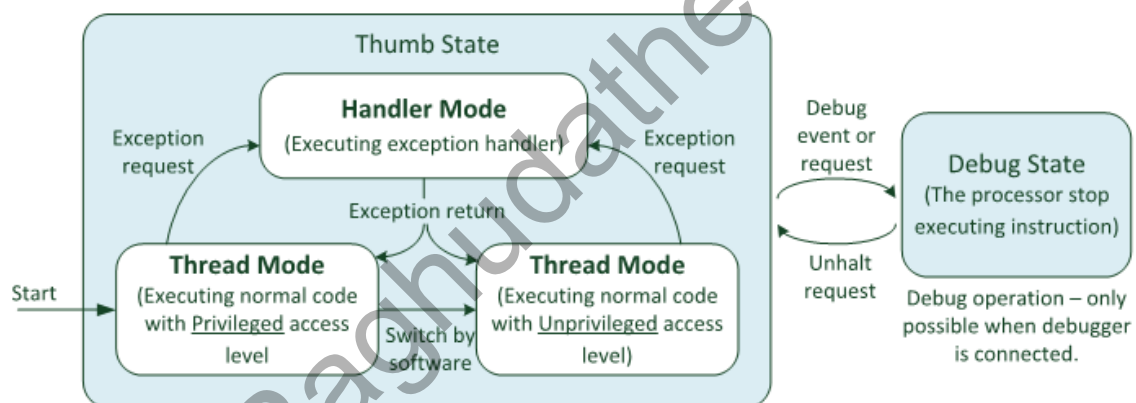


Figure 1: Operation states and modes

- **Access Level:**
 1. **privileged Access Level:** Can access all resources in the processor
 2. **Unprivileged Access Level:** Some memory regions are inaccessible, and a few operations cannot be used.
- In some documents, the unprivileged access level might also be referred as "User" state, a term inherited from ARM7TDMI.
- Software can switch the processor in privileged Thread mode to unprivileged Thread mode. However, it cannot switch itself back from unprivileged to privileged. If this is needed, the processor must use the exception mechanism to handle the switch.

- The separation of privileged and unprivileged access levels allows system designers to develop robust embedded systems by providing a mechanism to safeguard memory accesses to critical regions and by providing a basic security model.
- E.x: A system can contain an embedded OS kernel that executes in privileged access level, and application tasks which execute in unprivileged access level.
- In this way, we can set up memory access permissions using the Memory Protection Unit (MPU) to prevent an application task from corrupting memory and peripherals used by the OS kernel and other tasks.
- If an application task crashes, the remaining application tasks and the OS kernel can still continue to run.
- Besides the differences in memory access permission and access to several special instructions, the programmer's model of the privileged access level and unprivileged access level are almost the same. Note that almost all of the NVIC registers are privileged access only.

- **Operation States:**

1. **Debug State:** When the processor is halted (e.g., by the debugger, or after hitting a breakpoint), it enters debug state and stops executing instructions.
2. **Thumb State:**
 - If the processor is running program code (Thumb instructions), it is in the Thumb state.
 - Unlike classic ARM processors like ARM7TDMI, there is no ARM state because the Cortex-M processors do not support the ARM instruction set.

- **Operation Modes:**

1. **Handler Mode:**

- Processor enters when executing an exception handler such as an Interrupt Service Routine (ISR).
- When in handler mode, the processor always has privileged access level.

2. **Thread Mode:**

- Processor enters when executing normal application code.

- The processor can be either in privileged access level or unprivileged access level. This is controlled by a special register called “CONTROL.”
- Thread mode and Handler mode have very similar programmer’s models.
- However, Thread mode can switch to using a separate shadowed Stack Pointer (SP). This allows the stack memory for application tasks to be separated from the stack used by the OS kernel, thus allowing better system reliability.
- By default, the Cortex-M processors start in privileged Thread mode and in Thumb state.
- In many simple applications, there is no need to use the unprivileged Thread model and the shadowed SP at all as shown in figure 2 below. Unprivileged Thread model is not available in the Cortex-M0 processor, but is optional in the Cortex-M0+ processor.

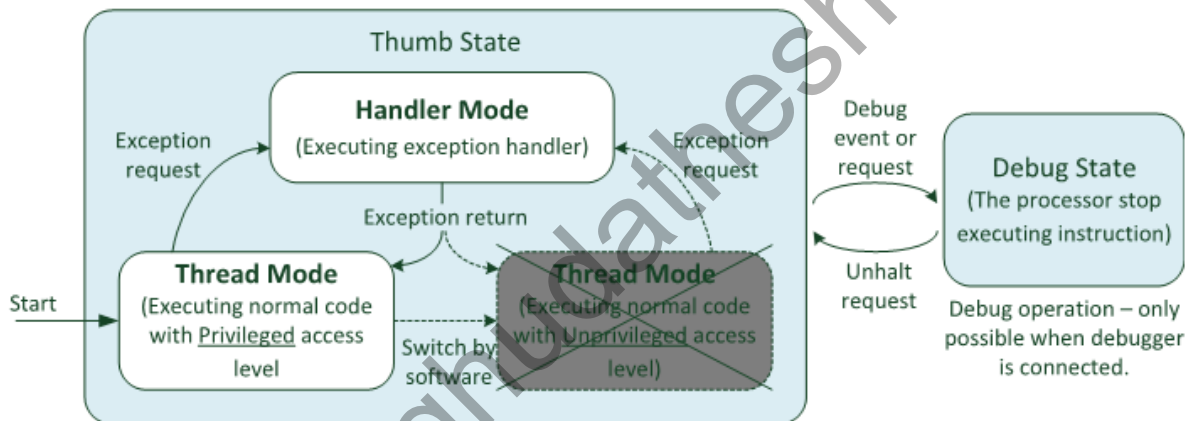


Figure 2: In simple applications, the unprivileged Thread mode can be unused

- The debug state is used for debugging operations only. This state is entered by a halt request from the debugger, or by debug events generated from debug components in the processor.
- This state allows the debugger to access or change the processor register values.
- The system memory, including peripherals inside and outside the processor, can be accessed by the debugger in either Thumb state or debug state.

Registers:

- The Cortex-M3 and Cortex-M4 processors have a number of registers inside the processor core to perform data processing and control. Most of these registers are grouped in a unit called the **register bank**.
- Each data processing instruction specifies the operation required, the source register(s), and the destination register(s) if applicable.
- In the ARM architecture, if data in memory is to be processed, it has to be loaded from the memory to registers in the register bank, processed inside the processor, and then written back to the memory, if needed. This is commonly called a “**load-store architecture**.”
- By having a sufficient number of registers in the register bank, this arrangement is easy to use, and allows efficient program code to be generated using C compilers.
- The register bank in the Cortex-M3 and Cortex-M4 processors has 16 registers.
- Thirteen of them are general purpose 32-bit registers, and the other three have special uses, as can be seen in Figure 3.

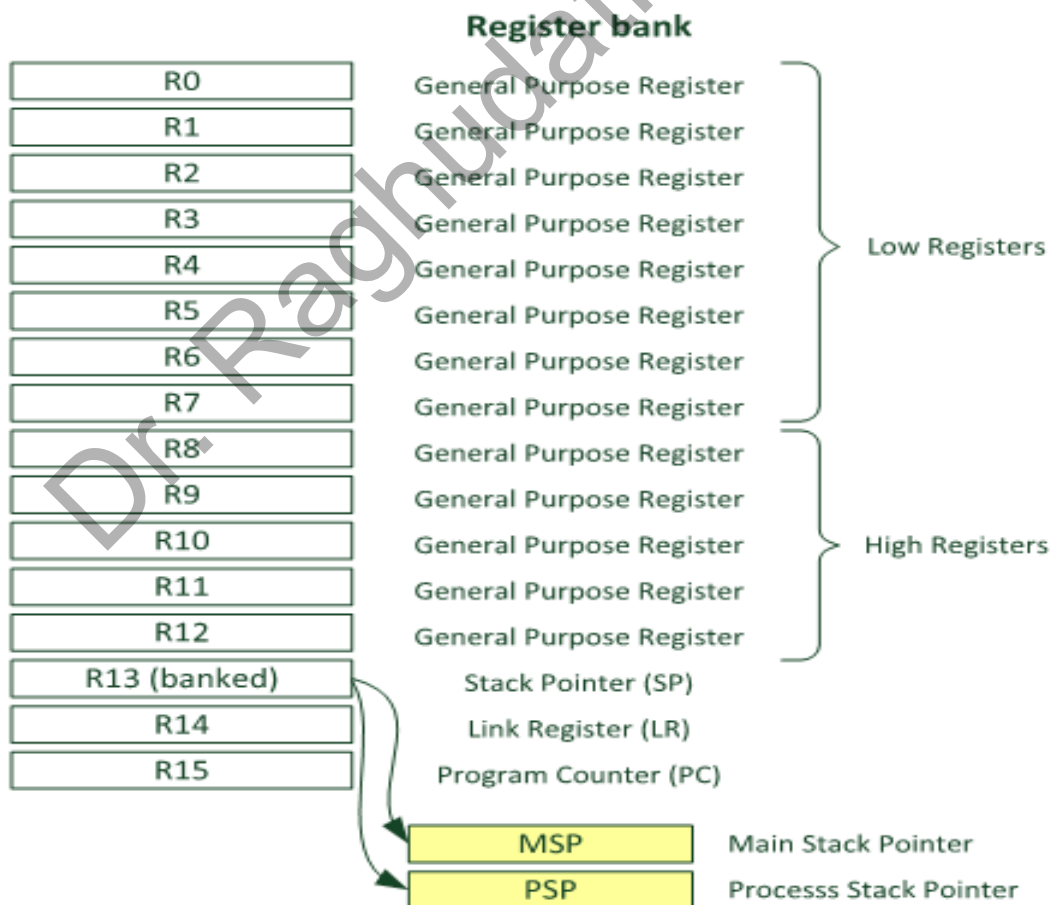


Figure 3: Registers in the Register Bank

- **R0 to R12:**
 - Registers R0 to R12 are general purpose registers.
 - The first eight (R0 to R7) are also called low registers.
 - Due to the limited available space in the instruction set, many 16-bit instructions can only access the low registers.
 - The high registers (R8 to R12) can be used with 32-bit instructions, and a few with 16-bit instructions, like MOV(move).
 - The initial values of R0 to R12 are undefined.

- **R13 - Stack Pointer (SP):**
 - R13 is called/referred as Stack Pointer.
 - It is used for accessing the stack memory via PUSH and POP operations.
 - Physically there are two different Stack Pointers:
 - **Main Stack Pointer** (MSP, or SP_main in some ARM documentation) is the default Stack Pointer. It is selected after reset, or when the processor is in Handler Mode.
 - **Process Stack Pointer** (PSP, or SP_process in some ARM documentation). The PSP can only be used in Thread Mode.
 - The selection of Stack Pointer is determined by a special register called CONTROL.
 - In normal programs, only one of these Stack Pointers will be visible.
 - Both MSP and PSP are 32-bit, but the lowest two bits of the Stack Pointers (either MSP or PSP) are always zero, and writes to these two bits are ignored.
 - In ARM Cortex-M processors, PUSH and POP are always 32-bit, and the addresses of the transfers in stack operations must be aligned to 32-bit word boundaries.
 - For most cases, it is not necessary to use the PSP if the application doesn't require an embedded OS.
 - Many simple applications can rely on the MSP completely.
 - The PSP is normally used when an embedded OS is involved, where the stack for the OS kernel and application tasks are separated.
 - The initial value of PSP is undefined, and the initial value of MSP is taken from the first word of the memory during the reset sequence.

- **R14 - Link Register (LR):**

- R14 is also called the Link Register (LR).
- This is used for holding the return address when calling a function or subroutine.
- At the end of the function or subroutine, the program control can return to the calling program and resume by loading the value of LR into the Program Counter (PC).
- When a function or subroutine call is made, the value of LR is updated automatically.
- If a function needs to call another function or subroutine, it needs to save the value of LR in the stack first. Otherwise, the current value in LR will be lost when the function call is made.
- During exception handling, the LR is also updated automatically to a special EXC_RETURN (Exception Return) value, which is then used for triggering the exception return at the end of the exception handler.
- Although the return address values in the Cortex-M processors are always even (bit 0 is zero because the instructions must be aligned to half-word addresses), bit 0 of LR is readable and writeable.
- Some of the branch/call operations require that bit zero of LR (or any register being used) be set to 1 to indicate Thumb state.

- **R15 - Program Counter (PC):**

- R15 is the Program Counter (PC).
- It is readable and writeable:
 - A read returns the current instruction address plus 4 (this is due to the pipeline nature of the design, and compatibility requirement with the ARM7TDMI processor).
 - Writing to PC (e.g., using data transfer/processing instructions) causes a branch operation.
- Since the instructions must be aligned to half-word or word addresses, the Least Significant Bit (LSB) of the PC is zero.
- However, when using some of the branch/memory read instructions to update the PC, you need to set the LSB of the new PC value to 1 to indicate the Thumb state. Otherwise, a fault exception can be triggered, as it indicates an attempt to switch to

use ARM instructions (i.e., 32-bit ARM instructions as in ARM7TDMI), which is not supported.

- In high-level programming languages (including C, C++), the setting of LSB in branch targets is handled by the compiler automatically.
- It is less common to use data processing instructions to update the PC. However, the value of PC is useful for accessing literal data stored in program memory. So you can frequently find memory read operations with PC as base address register with address offsets generated by immediate values in the instructions.

- **Special Registers:**

- Besides the registers in the register bank, there are several special registers as shown in figure 4.

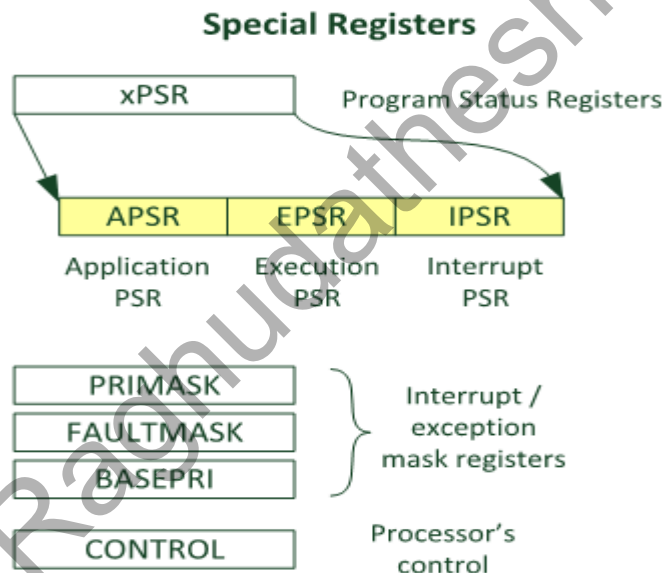


Figure 4: Special Registers

- These registers contain the processor status, define the operation states and interrupt/exception masking.
- In the development of simple applications with high level programming languages such as C, there are not many scenarios that require access to these registers. However, they are needed for development of an embedded OS, or when advanced interrupt masking features are needed.
- Special registers are not memory mapped, and can be accessed using special register access instructions such as MSR and MRS.

MRS <reg>, <special_reg>; Read special register into register

MSR <special_reg>, <reg>; write to special register

- CMSIS-Core also provides several C functions that can be used to access special registers.

- **Program Status Registers:**

- The Program Status Register is composed of three status registers:
 1. Application PSR (APSR)
 2. Execution PSR (EPSR)
 3. Interrupt PSR (IPSR)
- These three registers shown in figure 5 can be accessed as one combined register, referred to as xPSR in some documentation. In ARM assembler, when accessing xPSR shown in Figure 6, the symbol PSR is used.

	31	30	29	28	27	26:25	24	23:20	19:16	15:10	9	8	7	6	5	4:0
APSR	N	Z	C	V	Q				GE*							
IPSR												Exception Number				
EPSR						ICI/IT	T				ICI/IT					

*GE is available in ARMv7E-M processors such as the Cortex-M4. It is not available in the Cortex-M3 processor.

Figure 5: APSR, IPSR, and EPSR

	31	30	29	28	27	26:25	24	23:20	19:16	15:10	9	8	7	6	5	4:0
xPSR	N	Z	C	V	Q	ICI/IT		T		GE*	ICI/IT		Exception Number			

*GE is available in ARMv7E-M processors such as the Cortex-M4. It is not available in the Cortex-M3 processor.

Figure 6: Combined xPSR

- Below code snippet illustrates the accessing of all the three registers of PSR

MRS r0, PSR; Read the combined program status word

MSR PSR, r0; Write combined program state word

- Below code snippet illustrates the accessing individual PSR

MRS r0, APSR; Read Flag state into R0

MRS r0, IPSR; Read Exception/Interrupt state

MSR APSR, r0; Write Flag state

- Note:**

- The EPSR cannot be accessed by software code directly using MRS (read as zero) or MSR
- The IPSR is read only and can be read from combined PSR (xPSR).

- Figure 5 shows the definition of the various PSRs in ARMv7-M, and Table below lists the definition of the bit fields in the PSRs.

	31	30	29	28	27	26:25	24	23:20	19:16	15:10	9	8	7	6	5	4:0
APSR	N	Z	C	V	Q				GE*							
IPSR												Exception Number				
EPSR						ICI/IT		T			ICI/IT					

*GE is available in ARMv7E-M processors such as the Cortex-M4. It is not available in the Cortex-M3 processor.

Figure 7: APSR, IPSR, and EPSR

Table: Bit Fields in Program Status Registers

Bit	Description
N	Negative flag
Z	Zero flag
C	Carry (or NOT borrow) flag
V	Overflow flag
Q	Sticky saturation flag (not available in ARMv6-M)
GE[3:0]	Greater-Than or Equal flags for each byte lane (ARMv7E-M only; not available in ARMv6-M or Cortex®-M3).
ICI/IT	Interrupt-Continuable Instruction (ICI) bits, IF-THEN instruction status bit for conditional execution (not available in ARMv6-M).
T	Thumb state, always 1; trying to clear this bit will cause a fault exception.
Exception Number	Indicates which exception the processor is handling.

- CPSR in traditional ARM processors is shown in figure 8.

	31	30	29	28	27	26:25	24	23:20	19:16	15:10	9	8	7	6	5	4:0
ARM general (Cortex-A/R)	N	Z	C	V	Q	IT	J	Reserved	GE[3:0]	IT	E	A	I	F	T	M[4:0]
ARM7TDMI (ARMv4)	N	Z	C	V	Reserved								I	F	T	M[4:0]
ARMv7-M (Cortex-M3)	N	Z	C	V	Q	ICI/IT	T			ICI/IT		Exception Number				
ARMv7E-M (Cortex-M4)	N	Z	C	V	Q	ICI/IT	T		GE[3:0]	ICI/IT		Exception Number				
ARMv6-M (Cortex-M0)	N	Z	C	V			T									↑
Exception Number																

Figure 8: Comparing PSR of various ARM architectures

PRIMASK, FAULTMASK, and BASEPRI:

- The PRIMASK, FAULTMASK, and BASEPRI registers are all used for exception or interrupt masking.
- Each exception (including interrupts) has a priority level where a smaller number is a higher priority and a larger number is a lower priority.
- These special registers are used to mask exceptions based on priority levels.
- They can only be accessed in the privileged access level (in unprivileged state writes to these registers are ignored and reads return zero).
- By default, they are all zero, which means the masking (disabling of exception/interrupt) is not active.
- Figure 9 shows the programmer's model of these registers.

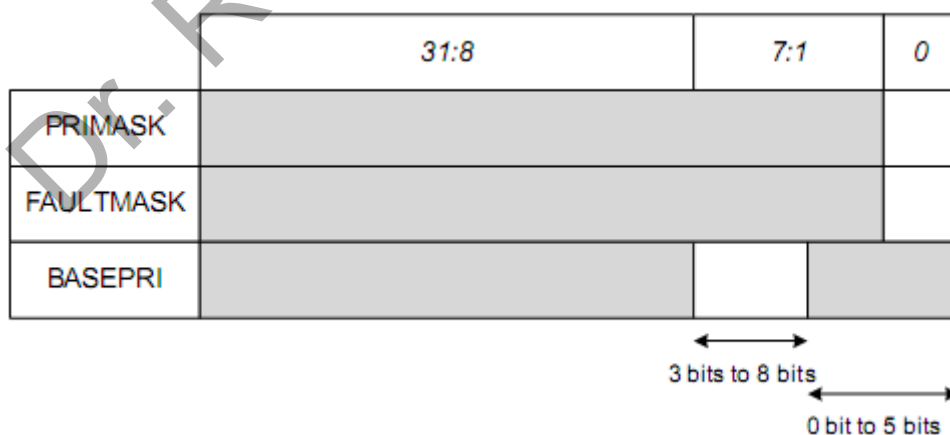


Figure 9: PRIMASK, FAULTMASK, and BASEPRI registers

PRIMASK Register:

- The PRIMASK register is a 1-bit wide interrupt mask register.

- When set, it blocks all exceptions (including interrupts) apart from the Non-Maskable Interrupt (NMI) and the HardFault exception.
- Effectively it raises the current exception priority level to 0, which is the highest level for a programmable exception/interrupt.
- The most common usage for PRIMASK is to disable all interrupts for a time critical process.
- After the time critical process is completed, the PRIMASK needs to be cleared to re-enable interrupts.

FAULTMASK Register:

- The FAULTMASK register is very similar to PRIMASK, but it also blocks the HardFault exception, which effectively raises the current exception priority level to -1.
- FAULTMASK can be used by fault handling code to suppress the triggering of further faults (only several types of them) during fault handling.
- E.x: FAULTMASK can be used to bypass MPU or suppress bus fault.
- This potentially makes it easier for fault handling code to carry out remedial actions.
- Unlike PRIMASK, FAULTMASK is cleared automatically at exception return.

BASEPRI Register:

- In order to allow more flexible interrupt masking, the ARMv7-M architecture also provides the BASEPRI, which masks exceptions or interrupts based on priority level.
- The width of the BASEPRI register depends on how many priority levels are implemented in the design, which is determined by the microcontroller vendors.
- Most Cortex-M3 or Cortex-M4 microcontrollers have eight programmable exception priority levels (3-bit width) or 16 levels, and in these cases the width of BASEPRI will be 3 bits or 4 bits, respectively.
- When BASEPRI is set to 0, it is disabled.
- When it is set to a non-zero value, it blocks exceptions (including interrupts) that have the same or lower priority level, while still allowing exceptions with a higher priority level to be accepted by the processor.

- CMSIS-Core provides a number of functions for accessing the PRIMASK, FAULTMASK, and BASEPRI registers in the C programming environment (note: these registers can only be accessed in the privileged access level).

```
x = __get_BASEPRI(); // Read BASEPRI register
x = __get_PRIMASK(); // Read PRIMASK register
x = __get_FAULTMASK(); // Read FAULTMASK register
__set_BASEPRI(x); // Set new value for BASEPRI
__set_PRIMASK(x); // Set new value for PRIMASK
__set_FAULTMASK(x); // Set new value for FAULTMASK
__disable_irq(); // Set PRIMASK, disable IRQ
__enable_irq(); // Clear PRIMASK, enable IRQ
```

- Alternatively, you can also access these exception masking registers with assembly code:

```
MRS r0, BASEPRI; Read BASEPRI register into R0
MRS r0, PRIMASK; Read PRIMASK register into R0
MRS r0, FAULTMASK; Read FAULTMASK register into R0
MSR BASEPRI, r0; Write R0 into BASEPRI register
MSR PRIMASK, r0; Write R0 into PRIMASK register
MSR FAULTMASK, r0; Write R0 into FAULTMASK register
```

- In addition, the Change Processor State (CPS) instructions allow the value of the PRIMASK and FAULTMASK to be set or cleared with a simple instruction.

```
CPSIE i; Enable interrupt (clear PRIMASK)
CPSID i; Disable interrupt (set PRIMASK)
CPSIE f; Enable interrupt (clear FAULTMASK)
CPSID f; Disable interrupt (set FAULTMASK)
```

- **Note:** The FAULTMASK and BASEPRI registers are not available in ARMv6-M (e.g., Cortex-M0).

CONTROL Register:

- The CONTROL register shown in figure 10 defines:
 - The selection of stack pointer (Main Stack Point/Process Stack Pointer)
 - Access level in Thread mode (Privileged/Unprivileged)

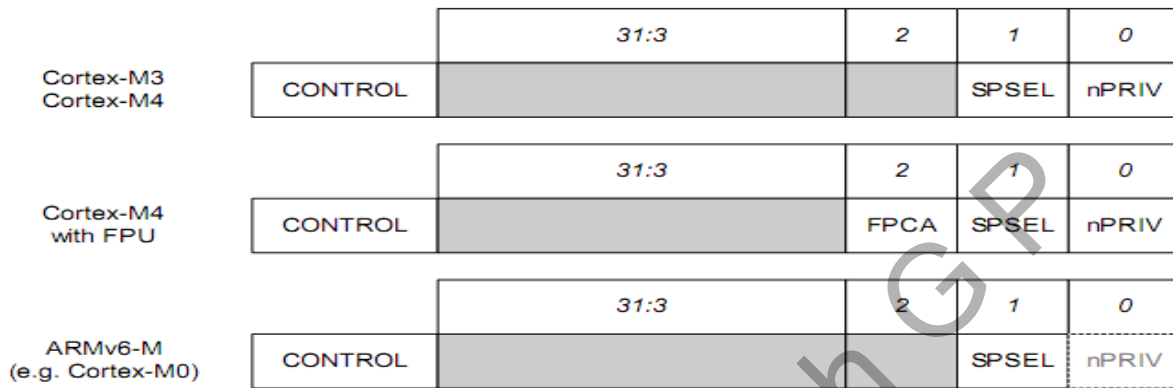


Figure 10: CONTROL register in Cortex-M3, Cortex-M4, Cortex-M4 with FPU. The bit nPRIV is not available in the Cortex-M0 and is optional in the Cortex-M0p processor

- The CONTROL register can only be modified in the privileged access level and can be read in both privileged and unprivileged access levels.
- The definition of each bit field in the CONTROL register is shown in Table below.

Table: Bit Fields in CONTROL Register

Bit	Function
nPRIV (bit 0)	Defines the privileged level in Thread mode: When this bit is 0 (default), it is privileged level when in Thread mode. When this bit is 1, it is unprivileged when in Thread mode. In Handler mode, the processor is always in privileged access level.
SPSEL (bit 1)	Defines the Stack Pointer selection: When this bit is 0 (default), Thread mode uses Main Stack Pointer (MSP). When this bit is 1, Thread mode uses Process Stack Pointer (PSP). In Handler mode, this bit is always 0 and write to this bit is ignored.
FPCA (bit 2)	Floating Point Context Active – This bit is only available in Cortex-M4 with floating point unit implemented. The exception handling mechanism uses this bit to determine if registers in the floating point unit need to be saved when an exception has occurred. When this bit is 0 (default), the floating point unit has not been used in the current context and therefore there is no need to save floating point registers. When this bit is 1, the current context has used floating point instructions and therefore need to save floating point registers. The FPCA bit is set automatically when a floating point instruction is executed. This bit is clear by hardware on exception entry. There are several options for handling saving of floating point registers.

- After reset, the CONTROL register is 0. This means the Thread mode uses the Main Stack Pointer as Stack Pointer and Thread mode has privileged accesses.
- Programs in privileged Thread mode can switch the Stack Pointer selection or switch to unprivileged access level by writing to CONTROL shown in figure 11. However, once nPRIV (CONTROL bit 0) is set, the program running in Thread can no longer access the CONTROL register.

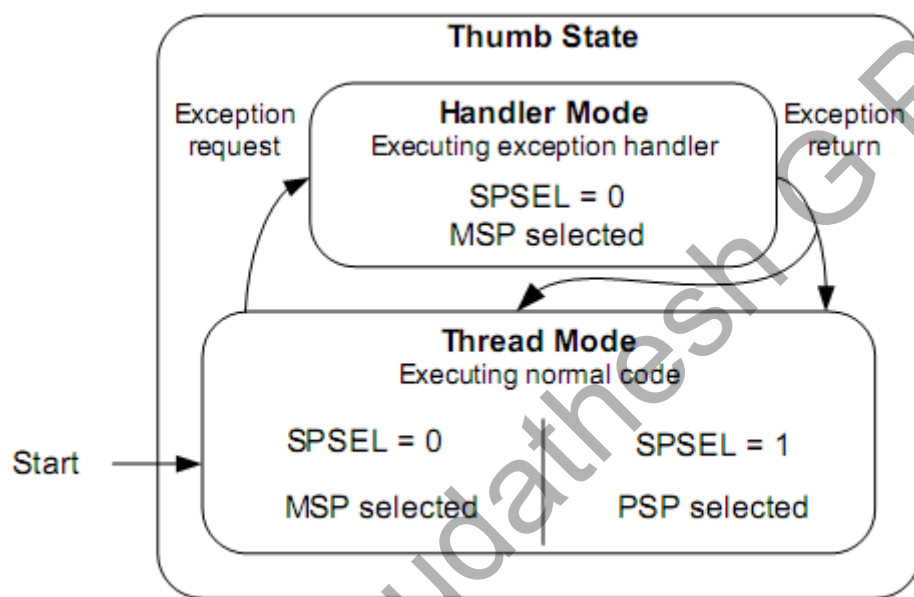


Figure 11: Stack Pointer selection

- A program in unprivileged access level cannot switch itself back to privileged access level. This is essential in order to provide a basic security usage model.
- Ex: an embedded system might contain untrusted applications running in unprivileged access level and the access permission of these applications must be restricted to prevent security breaches or to prevent an unreliable application from crashing the whole system.
- If it is necessary to switch the processor back to using privileged access level in Thread mode, then the exception mechanism is needed.
- During exception handling, the exception handler can clear the nPRIV bit shown in figure 12. When returning to Thread mode, the processor will be in privileged access level.

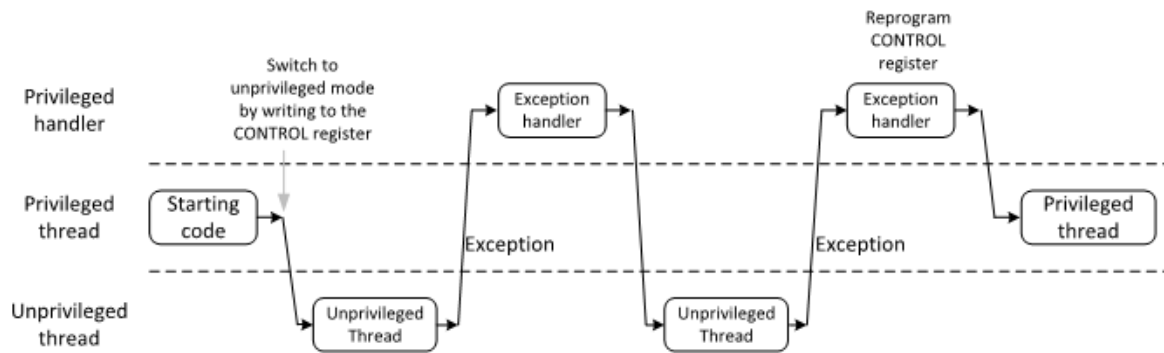


Figure 12: Switching between privileged thread mode and unprivileged thread mode

- When an embedded OS is used, the CONTROL register could be reprogrammed at each context switch to allow some application tasks to run with privileged access level and the others to run with unprivileged access level.
- The settings of nPRIV and SPSEL are orthogonal. Four different combinations of nPRIV and SPSEL are possible, although only three of them are commonly used in real world applications, as shown in Table below.

Table: Different Combinations of nPRIV and SPSEL

nPRIV	SPSEL	Usage Scenario
0	0	Simple applications – the whole application is running in privileged access level. Only one stack is used by the main program and interrupt handlers. Only the Main Stack Pointer (MSP) is used.
0	1	Applications with an embedded OS, with current executing task running in privileged Thread mode. The Process Stack Pointer (PSP) is selected in current task, and the MSP is used by OS Kernel and exception handlers.
1	1	Applications with an embedded OS, with current executing task running in unprivileged Thread mode. The Process Stack Pointer (PSP) is selected in current task, and the MSP is used by OS Kernel and exception handlers.
1	0	Thread mode tasks running with unprivileged access level and use MSP. This can be observed in Handler mode but is less likely to be used for user tasks because in most embedded OS, the stack for application tasks is separated from the stack used by OS kernel and exception handlers.

- In most simple applications without an embedded OS, there is no need to change the value of the CONTROL register. The whole application can run in privileged access level and use only the MSP as shown in figure 13.

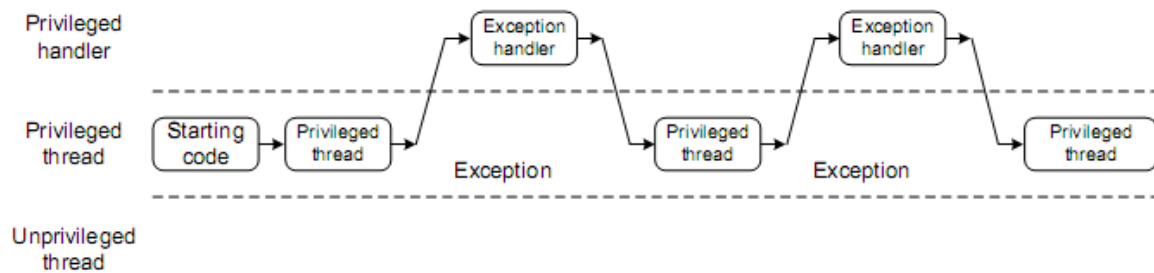


Figure 13: Simple applications do not require unprivileged Thread mode

- To access the CONTROL register in C, the following functions are available in CMSIS-compliant device-driver libraries:

```
x = __get_CONTROL(); // Read the current value of CONTROL
__set_CONTROL(x); // Set the CONTROL value to x
```

- To access the Control register in assembly, the MRS and MSR instructions are used:

```
MRS r0, CONTROL ; Read CONTROL register into R0
MSR CONTROL, r0 ; Write R0 into CONTROL register
```

- To detect if the current execution level is privileged by checking the value of IPSR and CONTROL:

```
int in_privileged(void)
{
    if (__get_IPSR() != 0) return 1; // True
    else
    if ((__get_CONTROL() & 0x1)==0) return 1; // True
    else return 0; // False
}
```

Floating Point Registers:

- The Cortex-M4 processor has an optional floating point unit. This provides additional registers for floating point data processing, as well as a Floating Point Status and Control Register (FPSCR) as shown in Figure 14

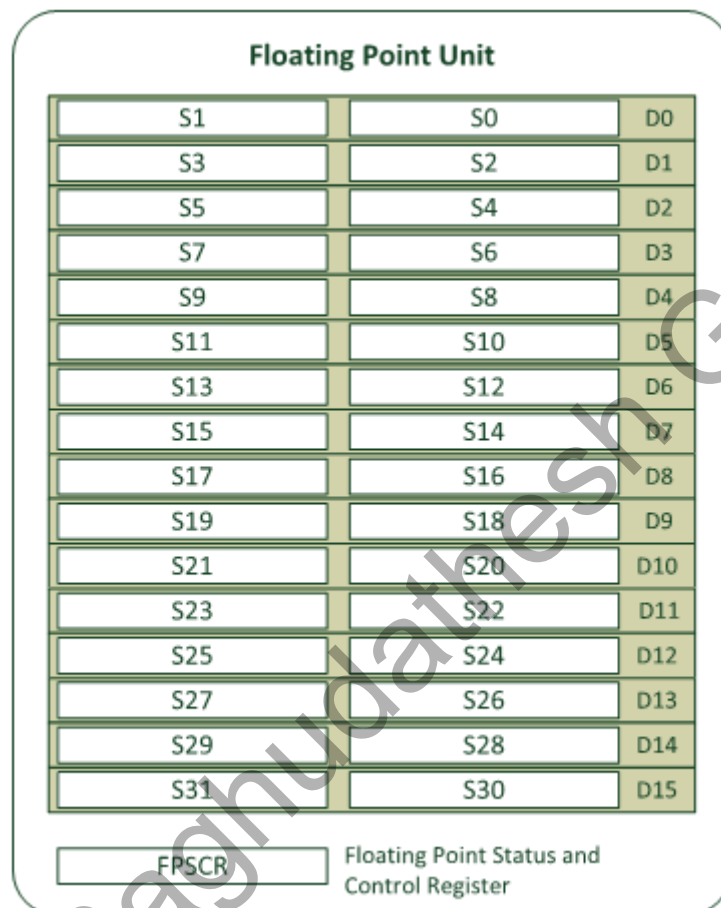


Figure 14: Registers in the floating point unit

S0 to S31/D0 to D15:

- Each of the 32-bit registers S0 to S31 (“S” for single precision) can be accessed using floating point instructions, or accessed as a pair, in the symbol of D0 to D15 (“D” for double-word/double-precision).
- For example, S1 and S0 are paired together to become D0, and S3 and S2 are paired together to become D1.
- Although the floating point unit in the Cortex-M4 does not support double precision floating point calculations, you can still use floating point instructions for transferring double precision data.

Floating point status and control register (FPSCR):

- The FPSCR contains various bit fields as shown in figure 15 for a couple of reasons:
- To define some of the floating point operation behaviours.
- To provide status information about the floating point operation results

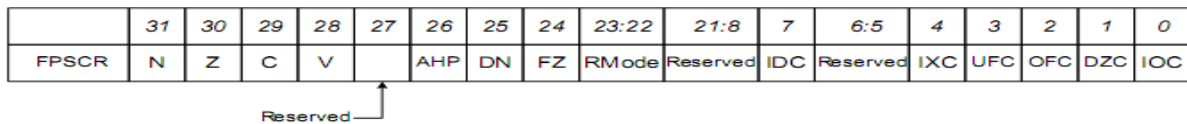


Figure 15: Bit field in FPSCR

- By default, the behaviour is configured to be compliant with IEEE 754 single precision operation.
- In normal applications there is no need to modify the settings of the floating point operation control.
- Table below lists the descriptions for the bit fields in FPSCR.
- **Note:** The exception bits in FPSCR can be used by software to detect abnormalities in floating point operations.

Table: Bit Fields in FPSCR

Bit	Description
N	Negative flag (update by floating point comparison operations)
Z	Zero flag (update by floating point comparison operations)
C	Carry/borrow flag (update by floating point comparison operations)
V	Overflow flag (update by floating point comparison operations)
AHP	Alternate half-precision control bit: 0 – IEEE half-precision format (default) 1 – Alternative half-precision format
DN	Default NaN (Not a Number) mode control bit: 0 – NaN operands propagate through to the output of a floating point operation (default) 1 – Any operation involving one or more NaN(s) returns the default NaN
FZ	Flush-to-zero model control bit: 0 – Flush-to-zero mode disabled (default). (IEEE 754 standard compliant) 1 – Flush-to-zero mode enabled
RMode	Rounding Mode Control field. The specified rounding mode is used by almost all floating-point instructions: 00 – Round to Nearest (RN) mode (default) 01 – Round towards Plus Infinity (RP) mode 10 – Round towards Minus Infinity (RM) mode 11 – Round towards Zero (RZ) mode
IDC	Input Denormal cumulative exception bit. Set to 1 when floating point exception occurred, clear by writing 0 to this bit. (Result not within normalized value range; see section 13.1.2.)
IXC	Inexact cumulative exception bit. Set to 1 when floating point exception occurred, clear by writing 0 to this bit.
UFC	Underflow cumulative exception bit. Set to 1 when floating point exception occurred, clear by writing 0 to this bit.
OFC	Overflow cumulative exception bit. Set to 1 when floating point exception occurred, clear by writing 0 to this bit.
DZC	Division by Zero cumulative exception bit. Set to 1 when floating point exception occurred, clear by writing 0 to this bit.
IOC	Invalid Operation cumulative exception bit. Set to 1 when floating point exception occurred, clear by writing 0 to this bit.

Memory-Mapped floating Point Unit Control Registers:

- In addition to the floating point register bank and FPSCR, the floating point unit also introduces several additional memory-mapped registers into the system.
- E.x: the Coprocessor Access Control Register (CPACR) is used to enable or disable the floating point unit.
- By default the floating point unit is disabled to reduce power consumption. Before using any floating point instructions, the floating point unit must be enabled by programming the CPACR register is as shown in Figure 16 below

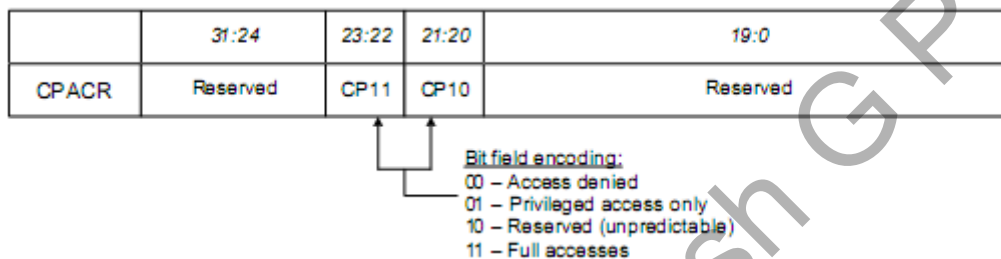


Figure 16: Bit field in CPACR

- In the C programming environment with a CMSIS-compliant device-driver:

```
SCB->CPACR |= 0xF << 20; // Enable full access to the FPU
```

- In assembly language programming environment, you can use the following code:

```
LDR R0,= 0xE000ED88 ; R0 set to address of CPACR
LDR R1,= 0x00F00000 ; R1 = 0xF << 20
LDR R2 [R0]; Read current value of CPACR
ORRS R2, R2, R1; Set bit
STR R2,[R0]; Write back modified value to CPACR
```

Behaviour of the application program status register (APSR):

- The APSR contains several groups of status flags:
 1. Status flags for integer operations (N-Z-C-V bits)
 2. Status flags for saturation arithmetic (Q bit)
 3. Status flags for SIMD operations (GE bits)

Integer Status Flags:

- The integer status flags are very similar to ALU status flags in many other processor architectures.
- These flags are affected by general data processing instructions, and essential for controlling conditional branches and conditional executions.
- In addition, one of the APSR flags, the C (Carry) bit, can also be used in add and subtract operations.
- There are four integer flags in the Cortex-M processors, shown in Table below.

Table: ALU Flags on the Cortex-M Processors

Flag	Descriptions
N (bit 31)	Set to bit[31] of the result of the executed instruction. When it is "1," the result has a negative value (when interpreted as a signed integer). When it is "0," the result has a positive value or equal zero.
Z (bit 30)	Set to "1" if the result of the executed instruction is zero. It can also be set to "1" after a compare instruction is executed if the two values are the same.
C (bit 29)	Carry flag of the result. For unsigned addition, this bit is set to "1" if an unsigned overflow occurred. For unsigned subtract operations, this bit is the inverse of the borrow output status. This bit is also updated by shift and rotate operations.
V (bit 28)	Overflow of the result. For signed addition or subtraction, this bit is set to "1" if a signed overflow occurred.

- A few examples of the ALU flag results are shown in Table below.

Table: ALU Flags Example

Operation	Results, Flags
0x70000000 + 0x70000000	Result = 0xE0000000, N= 1, Z=0, C = 0, V = 1
0x90000000 + 0x90000000	Result = 0x30000000, N= 0, Z=0, C = 1, V = 1
0x80000000 + 0x80000000	Result = 0x00000000, N= 0, Z=1, C = 1, V = 1
0x00001234 - 0x00001000	Result = 0x00000234, N= 0, Z=0, C = 1, V = 0
0x00000004 - 0x00000005	Result = 0xFFFFFFFF, N= 1, Z=0, C = 0, V = 0
0xFFFFFFFF - 0xFFFFFFF0	Result = 0x00000003, N= 0, Z=0, C = 1, V = 0
0x80000005 - 0x80000004	Result = 0x00000001, N= 0, Z=0, C = 1, V = 0
0x70000000 - 0xF0000000	Result = 0x80000000, N= 1, Z=0, C = 0, V = 1
0xA0000000 - 0xA0000000	Result = 0x00000000, N= 0, Z=1, C = 1, V = 0

- In the ARMv7-M and ARMv7E-M architecture, most of the 16-bit instructions affect these four ALU flags.
- In most of the 32-bit instructions one of the bits in the instruction encoding defines if the APSR flags should be updated or not.

- Note that some of these instructions do not update the V flag or the C flag.
- E.x: the MULS (multiply) instruction only changes the N flag and the Z flag.
- In addition to conditional branch or conditional execution code, the Carry bit of APSR can also be used to extend add and subtract operations to over 32bits.
- E.x: when adding two 64-bit integers together, we can use the carry bit from the lower 32-bit add operation as an extra input for the upper 32-bit add operation:

```
// Calculating Z = X + Y, where X, Y and Z are all 64-bit
Z[31:0] = X[31:0] + Y[31:0]; // Calculate lower word addition,
// carry flag get updated
Z[63:32] = X[63:32] + Y[63:32] + Carry; // Calculate upper
// word addition
```

- The N-Z-C-V flags are available in all ARM processors including the Cortex-M0 processor.

Q Status Flag:

- The Q is used to indicate an occurrence of saturation during saturation arithmetic operations or saturation adjustment operations.
- It is available in ARMv7-M (e.g., Cortex-M3 and Cortex-M4 processors), but not ARMv6-M (e.g., Cortex-M0 processor).
- After this bit is set, it remains set until a software write to the APSR clears the Q bit.
- Saturation arithmetic/adjustment operations do not clear this bit. As a result, you can use this bit to determine if saturation occurred at the end of a sequence of Saturation arithmetic/adjustment operations, without the need to check the saturation status during each step.
- Saturation arithmetic is useful for digital signal processing.
- In some cases, the destination register used to hold a calculation result might not have sufficient bit width and as a result, overflow or underflow occurs.
- If normal data arithmetic instructions are used, the MSB of the result would be lost and can cause a serious distortion in the output. Instead of just cutting off the MSB, saturation arithmetic forces the result to the maximum value (in case of overflow) or minimum value (in case of underflow) to reduce the impact of signal distortion as shown in figure 17

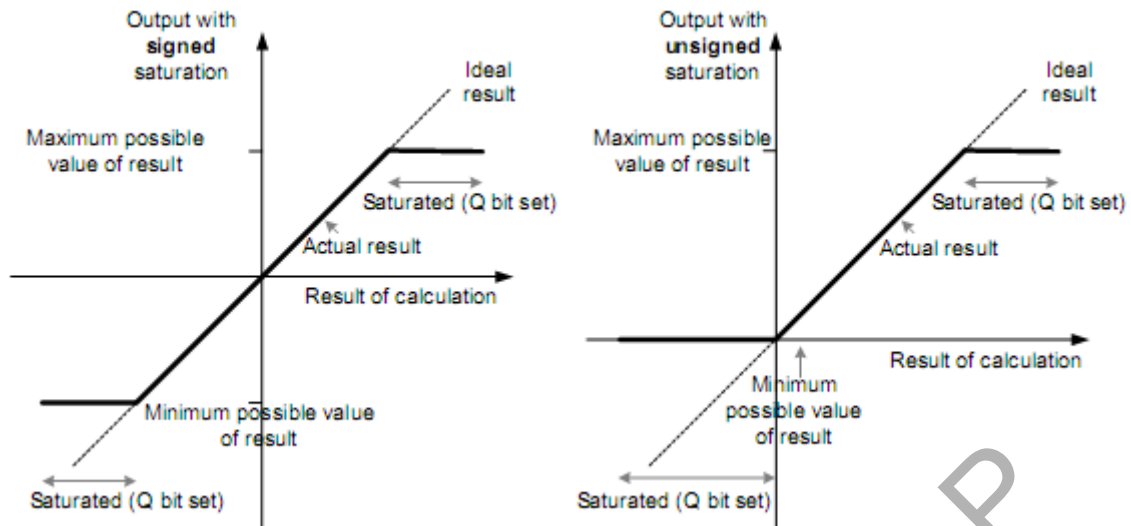


Figure 17: Signed saturation and unsigned saturation

- The actual maximum and minimum values that trigger the saturation depend on the instructions being used.
- In most cases, the instructions for saturation arithmetic are mnemonic starting with “Q,” for example “QADD16.”
- If saturation occurred, the Q bit is set; otherwise, the value of the Q bit is unchanged.
- The Cortex-M3 processor provides a couple of saturation adjustment instructions, and the Cortex-M4 provides a full set of saturation arithmetic instructions, as well as those saturation adjustment instructions available in the Cortex-M3 processor.

Memory System:

Memory System Features:

- The Cortex-M3 and Cortex-M4 processors have the following memory system features:
 1. **4GB linear address space** - With 32-bit addressing, the ARM processors can access up to 4GB of memory space. While many embedded systems do not need more than 1MB of memory, the 32-bit addressing capability ensures future upgrade and expansion possibilities. The Cortex-M3 and Cortex-M4 processors provide 32-bit buses using a generic bus protocol called AHB LITE. The bus allows connections to 32/16/8-bit memory devices with suitable memory interface controllers.
 2. **Architecturally defined memory map** - The 4GB memory space is divided into several regions for various predefined memory and peripheral uses. This allows the processor design to be optimized for performance. For example, the Cortex-M3 and

Cortex-M4 processors have multiple bus interfaces to allow simultaneous access from the CODE region for program code and data operations to SRAM or peripheral regions.

3. **Support for little endian and big endian memory systems** - The Cortex-M processors can work with either little endian or big endian memory systems. In practice, a microcontroller product is normally designed with just one endian configuration.
 4. **Bit band accesses (optional)** - When the bit-band feature is included (determined by microcontroller/System-on-Chip vendors), two 1MB regions in the memory map are bit addressable via two bit-band regions. This allows atomic access to individual bits in SRAM or peripheral address space.
 5. **Write buffer** - When a write transfer to a bufferable memory region will take multiple cycles, the transfer can be buffered by the internal write buffer in the Cortex-M3 or Cortex-M4 processor so that the processor can continue to execute the next instruction, if possible. This allows higher program execution speed.
 6. **Memory Protection Unit (Optional)** - The MPU is a programmable unit which defines access permissions for various memory regions. The MPU in the Cortex-M3 and Cortex-M4 processor supports eight programmable regions, and can be used with an embedded OS to provide a robust system.
 7. **Unaligned transfer support** - All processors supporting ARMv7-M architecture (including Cortex-M3 and Cortex-M4 processors) support unaligned data transfers.
- The bus interfaces on the Cortex-M processors are generic bus interfaces, and can be connected to different types and sizes of memory via different memory controllers.
 - The memory systems in microcontrollers often contain two or more types of memories: flash memory for program code, static RAM (SRAM) for data, and in some cases Electrical Erasable Read Only Memory (EEPROM).
 - In most cases, these memories are on-chip and the actual memory interface details are transparent to software developers. Hence, software developers only need to know the address and size of the program memory and SRAM.

Memory Map:

- The 4GB address space of the Cortex-M processors is partitioned into a number of memory regions as shown in figure 19.

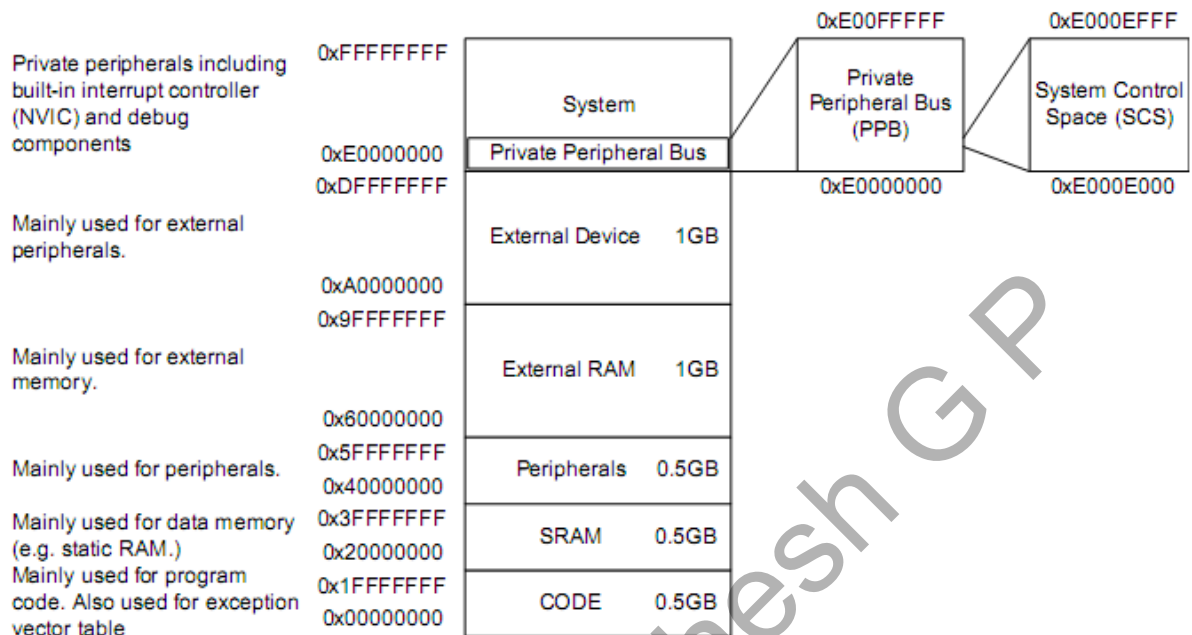


Figure 19: Memory Map

- The partitioning is based on typical usages so that different areas are designed to be used primarily for:
 1. Program code accesses (e.g., CODE region)
 2. Data accesses (e.g., SRAM region)
 3. Peripherals (e.g., Peripheral region)
 4. Processor's internal control and debug components (e.g., Private Peripheral Bus)
- The architecture also allows high flexibility to allow memory regions to be used for other purposes.
- E.x: programs can be executed from the CODE as well as the SRAM region, and a microcontroller can also integrate SRAM blocks in CODE region.
- In practice, many microcontroller devices only use a small portion of each region or program flash, SRAM, and peripherals. Some of the regions can be unused.
- Different microcontrollers have different memory sizes and peripheral address locations. This information is usually outlined in user manuals or datasheets from microcontroller vendors.
- The memory map arrangement is consistent between all of the Cortex-M processors.

Stack Memory:

- The Cortex -M processors need stack memory to operate and have stack pointers (R13).
- Stack is a kind of memory usage mechanism that allows a portion of memory to be used as Last-In-First-Out data storage buffer.
- ARM processors use the main system memory for stack memory operations, and have the PUSH instruction to store data in stack and the POP instruction to retrieve data from stack.
- The current selected stack pointer is automatically adjusted for each PUSH and POP operation.
- Stack can be used for:
 1. Temporary storage of original data when a function being executed needs to use registers (in the register bank) for data processing. The values can be restored at the end of the function so the program that called the function will not lose its data.
 2. Passing of information to functions or subroutines.
 3. For storing local variables.
 4. To hold processor status and register values in the case of exceptions such as an interrupt.
- The Cortex-M processors use a stack memory model called “full-descending stack.” When the processor is started, the SP is set to the end of the memory space reserved for stack memory.
- For each PUSH operation, the processor first decrements the SP, then stores the value in the memory location pointed by SP.
- During operations, the SP points to the memory location where the last data was pushed to the stack as shown in figure 20

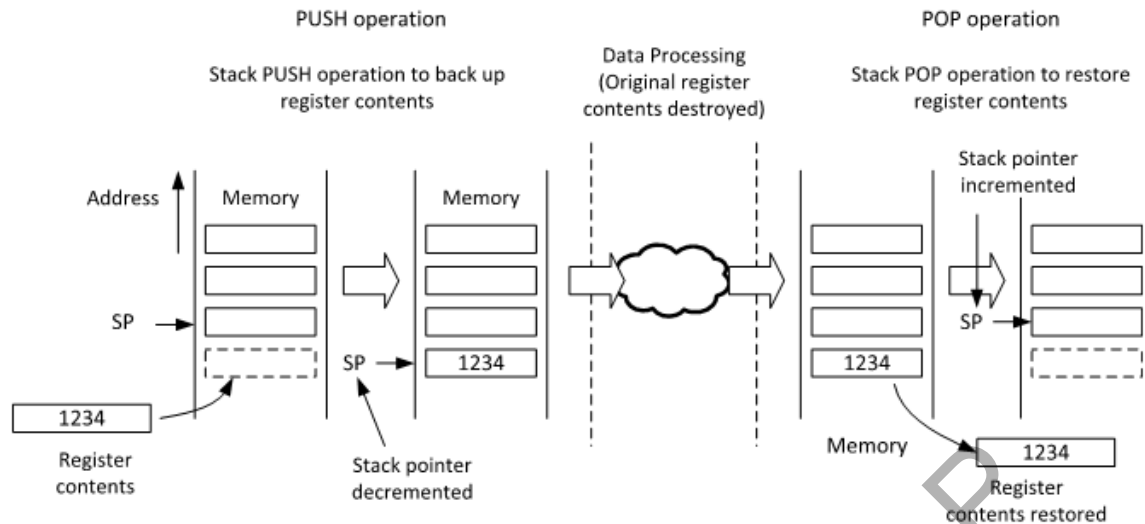


Figure 20: Stack PUSH and POP

- In a POP operation, the value of the memory location pointed by SP is read, and then the value of SP is incremented automatically.
- The most common uses for PUSH and POP instructions are to save contents of register banks when a function/subroutine call is made.
- At the beginning of the function call, the contents of some of the registers can be saved to the stack using the PUSH instruction, and then restored to their original values at the end of the function using the POP instruction.
- E.x: In Figure 21 a simple function/subroutine named function1 is called from the main program. Since function1 needs to use and modify R4, R5, and R6 for data processing, and these registers hold values that the main program need later, they are saved to the stack using PUSH, and restored using POP at the end of function1.

Main Program

```
...
; R4 = X, R5 = Y, R6 = Z
BL  function1
```

Subroutine

```
function1
```

```
PUSH    {R4} ; store R4 to stack & adjust SP
PUSH    {R5} ; store R5 to stack & adjust SP
PUSH    {R6} ; store R6 to stack & adjust SP
... ; Executing task (R4, R5 and R6
    ; could be changed)
POP     {R4} ; restore R4 and SP re-adjusted
POP     {R5} ; restore R5 and SP re-adjusted
POP     {R6} ; restore R6 and SP re-adjusted
BX      LR   ; Return
```

```
; Back to main program
; R4 = X, R5 = Y, R6 = Z
... ; next instructions
```

Figure 21: Simple PUSH and POP usage in functions e one register in each stack operation

- In this way, the program code that called the function will not lose any data and can continue to execute.
- Note that for each PUSH (store to memory) operation, there must be a corresponding POP (read from memory), and the address of the POP should match that of the PUSH operation.
- Each PUSH and POP instruction can transfer multiple data to/from the stack memory. This is shown in Figure 22.

Main Program

```
...
; R4 = X, R5 = Y, R6 = Z
BL    function1
```

Subroutine

function1

```
PUSH    {R4-R6, LR} ; Save registers
                ; including link register
... ; Executing task (R4, R5 and R6
    ; could be changed)
POP     {R4-R6, PC} ; Restore registers and
                ; return
```

```
; Back to main program
; R4 = X, R5 = Y, R6 = Z
... ; next instructions
```

Figure 22: Combining stack POP and return

- Since the registers in the register bank are 32 bits, each memory transfer generated by stack PUSH and stack POP transfers at least 1 word (4 bytes) of data, and the addresses are always aligned to 4-byte boundaries. The lowest two bits of the SP are always zero.
- Physically there are two stack pointers in the Cortex-M processors:
 1. **Main Stack Pointer (MSP)** - This is the default stack pointer used after reset, and is used for all exception handlers.
 2. **Process Stack Pointer (PSP)** - This is an alternate stack point that can only be used in Thread mode. It is usually used for application tasks in embedded systems running an embedded OS.
- The selection between MSP and PSP can be controlled by the value of SPSEL in bit 1 of the CONTROL register.
- If this bit is 0, Thread mode uses MSP for the stack operation. Otherwise, Thread mode uses the PSP. In addition, during exception return from Handler mode to Thread mode,

the selection can be controlled by the value of EXC_RETURN (exception return) value. In that case the value of SPSEL will be updated by the processor hardware accordingly.

- In simple applications without an OS, both Thread mode and Handler mode can use MSP only. This is shown in figure 24 After an interrupt event is triggered, the processor first pushes a number of registers into the stack before entering the Interrupt Service Routine (ISR). This register state saving operation is called “Stacking,” and at the end of the ISR, these registers are restored to the register bank and this operation is called “Unstacking.”

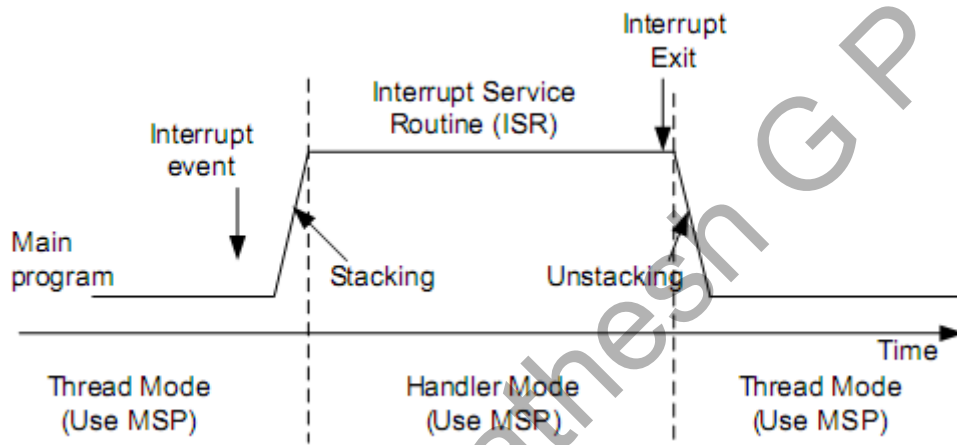


Figure 23: SPSEL= 0. Both Thread Level and Handler use the Main Stack Pointer

- When embedded systems use an embedded OS, they often use separate memory areas for application stack and the kernel stack.
- As a result, the PSP is used and switching of SP selection takes place in exception entry and exception exit. This is shown in Figure 24.

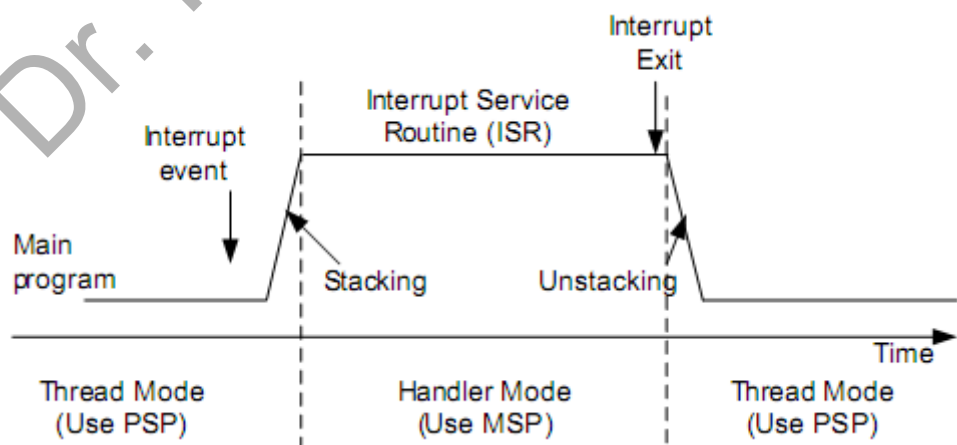


Figure 24: SPSEL = 1. Thread Level uses the Process Stack and Handler uses the Main Stack

- Note that the automatic “Stacking” and “Unstacking” stages use PSP. The separating stack arrangement can prevent a stack corruption or error in an application task from damaging the stack use by the OS. It also simplifies the OS design and hence allows faster context switching.
- Although only one of the SPs is visible at a time (when using SP or R13 to access it), it is possible to read/write directly to the MSP and PSP, without any confusion over which SP/R13 you are referring to. Provided that you are in privileged level, you can access MSP and PSP using the following CMSIS functions:

```
x = __get_MSP(); // Read the value of MSP
__set_MSP(x); // Set the value of MSP
x = __get_PSP(); // Read the value of PSP
__set_PSP(x); // Set the value of PSP
```

- In general it is not recommended to change the value of the current selected SP in a C function, as part of the stack memory could be used for storing local variables or other data. To access MSP and PSP in assembly code, you can use the MSR and MRS instructions:

```
MRS R0, MSP ; Read Main Stack Pointer to R0
MSR MSP, R0 ; Write R0 to Main Stack Pointer
MRS R0, PSP ; Read Process Stack Pointer to R0
MSR PSP, R0 ; Write R0 to Process Stack Pointer
```

- Most application code does not need to access MSP and PSP explicitly. Access to MSP and PSP is often required for embedded OSs.
- For example, by reading the PSP value using an MRS instruction, the OS can read data pushed to the stack from API calls in application tasks (such as register contents before execution of an SVC instruction). Also, the value of PSP is updated by context switching code in the OS during context switching.
- After power up, the processor hardware automatically initializes the MSP by reading the vector table.
- The PSP is not initialized automatically and must be initialized by the software before being used.

Memory protection unit (MPU):

- The MPU is optional in the Cortex-M3 and Cortex-M4 processors. Therefore not all Cortex-M3 or Cortex-M4 microcontrollers have the MPU feature.
- In the majority of applications, the MPU is not used and can be ignored. In embedded systems that require high reliability, the MPU can be used to protect memory regions by means of defining access permissions in privileged and unprivileged access states.
- The MPU is programmable, and the MPU design in the Cortex-M3 and Cortex-M4 processors supports eight programmable regions.
- The MPU can be used in different ways. In some cases the MPU is controlled by an embedded OS, and memory permissions are configured for each task. In other cases the MPU is configured just to protect a certain memory region; for example, to make a memory range read only.

Exceptions and Interrupts:

- Exceptions are events that cause changes to program flow.
- When one happens, the processor suspends the current executing task and executes a part of the program called the exception handler. After the execution of the exception handler is completed, the processor then resumes normal program execution.
- In the ARM architecture, interrupts are one type of exception. Interrupts are usually generated from peripheral or external inputs, and in some cases they can be triggered by software.
- The exception handlers for interrupts are also referred to as Interrupt Service Routines (ISR).
- In Cortex-M processors, there are a number of exception sources.
- Exceptions are processed by the NVIC. The NVIC can handle a number of Interrupt Requests (IRQs) and a Non-Maskable Interrupt (NMI) request.
- Usually IRQs are generated by on-chip peripherals or from external interrupt inputs through I/O ports.
- The NMI could be used by a watchdog timer or brownout detector (a voltage monitoring unit that warns the processor when the supply voltage drops below a certain level).

- Inside the processor there is also a timer called SysTick, which can generate a periodic timer interrupt request, which can be used by embedded OSs for timekeeping, or for simple timing control in applications that don't require an OS.
- The processor itself is also a source of exception events. These could be fault events that indicate system error conditions, or exceptions generated by software to support embedded OS operations. The exception types are listed in Table below.

Table: Exception Types

Exception Number	CMSIS Interrupt Number	Exception Type	Priority	Function
1	—	Reset	−3 (Highest)	Reset
2	−14	NMI	−2	Non-Maskable interrupt
3	−13	HardFault	−1	All classes of fault, when the corresponding fault handler cannot be activated because it is currently disabled or masked by exception masking
4	−12	MemManage	Settable	Memory Management fault; caused by MPU violation or invalid accesses (such as an instruction fetch from a non-executable region)
5	−11	BusFault	Settable	Error response received from the bus system; caused by an instruction prefetch abort or data access error
6	−10	Usage fault	Settable	Usage fault; typical causes are invalid instructions or invalid state transition attempts (such as trying to switch to ARM state in the Cortex-M3)
7–10	—	—	—	Reserved
11	−5	SVC	Settable	Supervisor Call via SVC instruction
12	−4	Debug monitor	Settable	Debug monitor – for software based debug (often not used)
13	—	—	—	Reserved
14	−2	PendSV	Settable	Pendable request for System Service
15	−1	SYSTICK	Settable	System Tick Timer
16–255	0–239	IRQ	Settable	IRQ input #0–239

- Each exception source has an exception number. Exception numbers 1 to 15 are classified as system exceptions, and exceptions 16 and above are for interrupts.
- The design of the NVIC in the Cortex-M3 and Cortex-M4 processors can support up to 240 interrupt inputs. However, in practice the number of interrupt inputs implemented in the design is far less, typically in the range of 16 to 100. In this way the silicon size of the design can be reduced, which also reduces power consumption.
- The exception number is reflected in various registers, including the IPSR, and it is used to determine the exception vector addresses.
- Exception vectors are stored in a vector table, and the processor reads this table to determine the starting address of an exception handler during the exception entrance sequence.
- Reset is a special kind of exception. When the processor exits from a reset, it executes the reset handler in Thread mode (rather than Handler mode as in other exceptions). Also the exception number in IPSR is read as zero.

Nested Vectored Interrupt Controller (NVIC):

- The NVIC is a part of the Cortex-M processor. It is programmable and its registers are located in the System Control Space (SCS) of the memory map as shown in figure 25

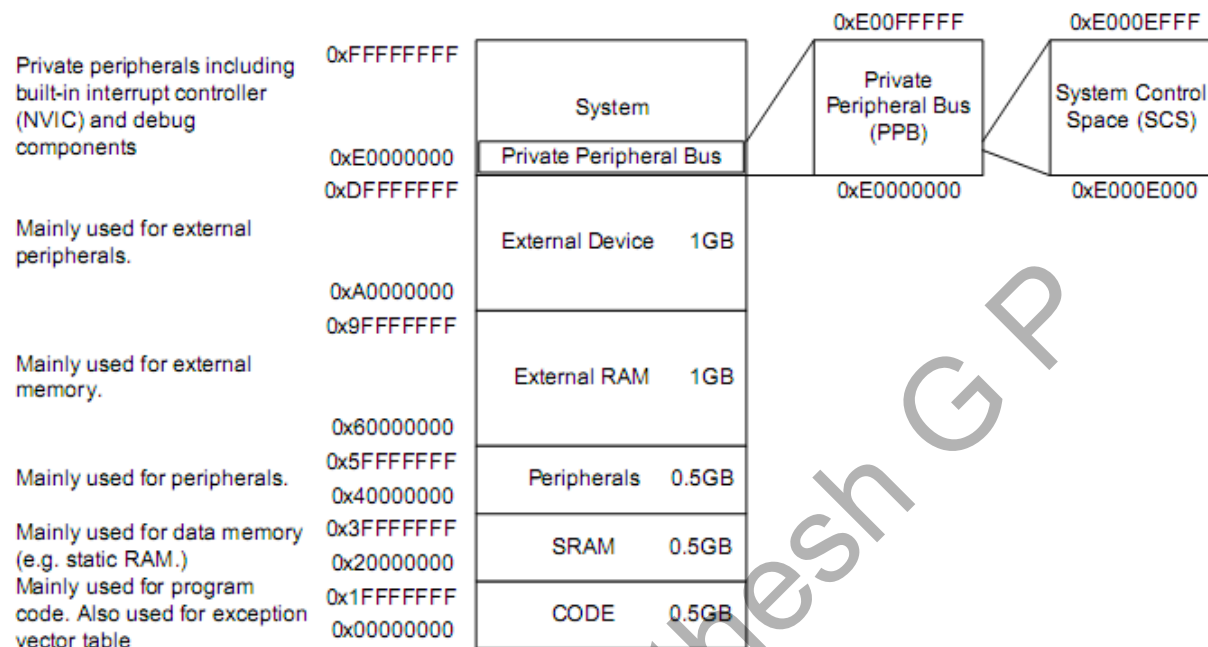


Figure 25: Memory Map

- The NVIC handles the exceptions and interrupt configurations, prioritization, and interrupt masking.
- The NVIC has the following features:

1. Flexible exception and interrupt management:

- Each interrupt (apart from the NMI) can be enabled or disabled and can have its pending status set or cleared by software.
- The NVIC can handle various types of interrupt sources:
 - **Pulsed interrupt request** - the interrupt request is at least one clock cycle long. When the NVIC receives a pulse at its interrupt input, the pending status is set and held until the interrupt gets serviced.
 - **Level triggered interrupt request** - the interrupt source holds the request high until the interrupt is serviced.
- The signal level at the NVIC input is active high. However, the actual external interrupt input on the microcontroller could be designed differently and is converted to an active high signal level by on-chip logic.

2. Nested Exception/Interrupt Support:

- Each exception has a priority level. Some exceptions, such as interrupts, have programmable priority levels and some others (e.g., NMI) have a fixed priority level.
- When an exception occurs, the NVIC will compare the priority level of this exception to the current level.
- If the new exception has a higher priority, the current running task will be suspended. Some of the registers will be stored on the stack memory, and the processor will start executing the exception handler of the new exception. This process is called “preemption.”
- When the higher priority exception handler is complete, it is terminated with an exception return operation and the processor automatically restores the registers from stack and resumes the task that was running previously.
- This mechanism allows nesting of exception services without any software overhead.

3. Vectored Exception/Interrupt Entry:

- When an exception occurs, the processor will need to locate the starting point of the corresponding exception handler.
- Traditionally, in ARM processors such as the ARM7TDMI, software handles this step. The Cortex-M processors automatically locate the starting point of the exception handler from a vector table in the memory.
- As a result, the delays from the start of the exception to the execution of the exception handlers are reduced.

4. Interrupt Masking:

- The NVIC in the Cortex-M3 and Cortex-M4 processors provide several interrupt masking registers such as the PRIMASK special register.
- Using the PRIMASK register you can disable all exceptions, excluding HardFault and NMI.
- This masking is useful for operations that should not be interrupted, like time critical control tasks or real-time multimedia codecs.
- Alternatively you can also use the BASEPRI register to select mask exceptions or interrupts which are below a certain priority level.
- The CMSIS-Core provides a set of functions to make it easy to access various interrupt control functions.

- The flexibility and capability of the NVIC also make the Cortex-M processors very easy to use, and provide better a system response by reducing the software overhead in interrupt processing, which also leads to smaller code size.

Vector Table:

- When an exception event takes place and is accepted by the processor core, the corresponding exception handler is executed.
- To determine the starting address of the exception handler, a vector table mechanism is used.
- The vector table is an array of word data inside the system memory, each representing the starting address of one exception type as shown in figure 26
- The vector table is relocatable and the relocation is controlled by a programmable register in the NVIC called the Vector Table Offset Register (VTOR).
- After reset, the VTOR is reset to 0; therefore, the vector table is located at address 0x0 after reset.

Exception Type	CMSIS Interrupt Number	Address Offset	Vectors
18 - 255	2 - 239	0x48 – 0x3FF	IRQ #2 - #239 1
17	1	0x44	IRQ #1 1
16	0	0x40	IRQ #0 1
15	-1	0x3C	SysTick 1
14	-2	0x38	PendSV 1
NA	NA	0x34	Reserved
12	-4	0x30	Debug Monitor 1
11	-5	0x2C	SVC 1
NA	NA	0x28	Reserved
NA	NA	0x24	Reserved
NA	NA	0x20	Reserved
NA	NA	0x1C	Reserved
6	-10	0x18	Usage fault 1
4	-11	0x14	Bus Fault 1
4	-12	0x10	MemManage Fault 1
3	-13	0x0C	HardFault 1
2	-14	0x08	NMI 1
1	NA	0x04	Reset 1
NA	NA	0x00	Initial value of MPS

Figure 26: Exception types (LSB of exception vectors should be set to 1 to indicate Thumb state)

- E.x: if the reset is exception type 1, the address of the reset vector is 1 times 4 (each word is 4 bytes), which equals 0x00000004, and the NMI vector (type 2) is located at $2 \times 4 = 0x00000008$. The address 0x00000000 is used to store the starting value of the MSP.
- The LSB of each exception vector indicates whether the exception is to be executed in the Thumb state. Since the Cortex-M processors can support only Thumb instructions, the LSB of all the exception vectors should be set to 1.

Fault Handling:

- Several types of exceptions in the Cortex-M3 and Cortex-M4 processors are fault handling exceptions.
- Fault exceptions are triggered when the processor detects an error such as the execution of an undefined instruction, or when the bus system returns an error response to a memory access.

- The fault exception mechanism allows errors to be detected quickly, and potentially allows the software to carry out remedial actions as shown in figure 27

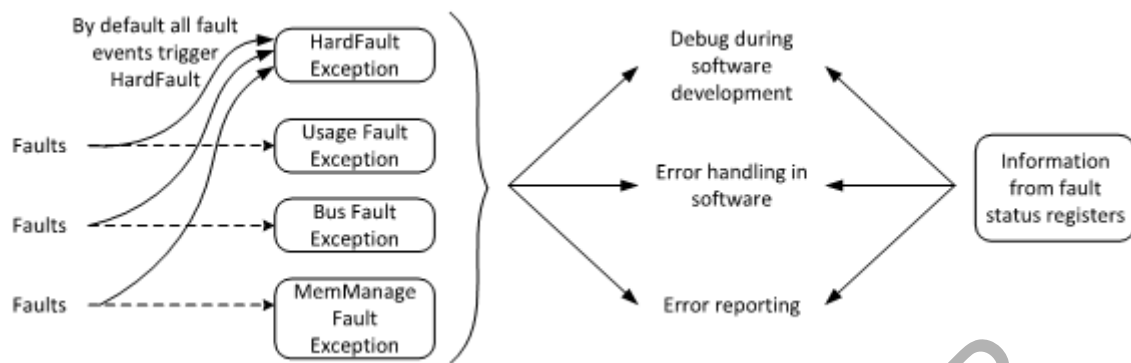


Figure 27: Fault exceptions usages

- By default the Bus Fault, Usage Fault, and Memory Management Fault are disabled and all fault events trigger the HardFault exception.
- However, the configurations are programmable and you can enable the three programmable fault exceptions individually to handle different types of faults.
- The HardFault exception is always enabled.
- Fault exceptions can also be useful for debugging software issues.
- E.x: the fault handler can automatically collect information and report to the user or other systems that an error has occurred and provide debug information.
- A number of fault status registers are available in the Cortex-M3 and Cortex-M4 processors, which provide hints about the error sources. Software developers can also examine these fault status registers using the debugger during software development.

Reset and Reset Sequence:

- In typical Cortex-M microcontrollers, there can be three types of reset:
 - **Power on reset** - reset everything in the microcontroller. This includes the processor and its debug support component and peripherals.
 - **System reset** - reset just the processor and peripherals, but not the debug support component of the processor.
 - **Processor reset** - reset the processor only.
- During system debug or processor reset operations, the debug components in the Cortex-M3 or Cortex-M4 processors are not reset so that the connection between the debug host (e.g., debugger software running on a computer) and the microcontroller

can be maintained. The debug host can generate a system reset or processor reset via a register in the System Control Block (SCB).

- The duration of Power on reset and System reset depends on the microcontroller design. In some cases the reset lasts a number of milli seconds as the reset controller needs to wait for a clock source such as a crystal oscillator to stabilize.
- After reset and before the processor starts executing the program, the Cortex-M processors read the first two words from the memory as shown in figure 28.

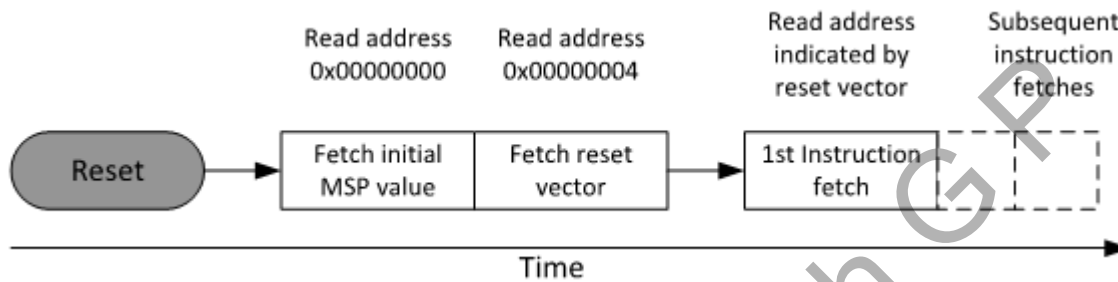


Figure 28: Reset sequence

- The beginning of the memory space contains the vector table, and the first two words in the vector table are the initial value for the Main Stack Pointer (MSP), and the reset vector, which is the starting address of the reset handler.
- After these two words are read by the processor, the processor then sets up the MSP and the Program Counter (PC) with these values.
- The setup of the MSP is necessary because some exceptions such as the NMI or HardFault handler could potentially occur shortly after the reset, and the stack memory and hence the MSP will then be needed to push some of the processor status to the stack before exception handling.
- Note that for most C development environments, the C startup code will also update the value of the MSP before entering the main program `main()`.
- As the stack operations in the Cortex-M3 or Cortex-M4 processors are based on full descending stack (SP decrement before store), the initial SP value should be set to the first memory after the top of the stack region.
- E.x., if you have a stack memory range from 0x20007C00 to 0x20007FFF (1Kbytes), the initial stack value should be set to 0x20008000, as shown in Figure 29.

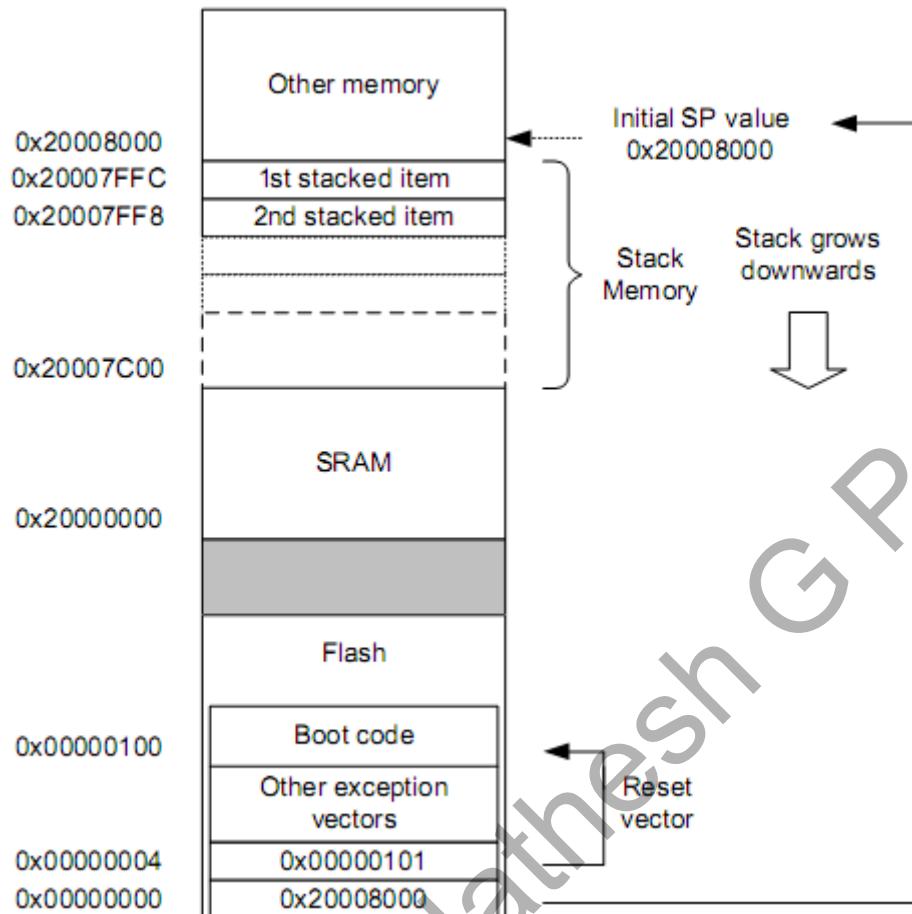


Figure 29: Initial Stack Pointer value and Initial Program Counter value example

- Notice that in the Cortex-M processors, vector addresses in the vector table should have their LSB set to 1 to indicate that they are Thumb code.
- For that reason the example in Figure 29 has 0x101 in the reset vector, whereas the boot code starts at address 0x100. After the reset vector is fetched, the Cortex-M processor can then start to execute the program from the reset vector address and begin normal operations.

Definitions:

- **Memory Protection Unit (MPU)** is a programmable unit that allows privileged software, typically an OS kernel, to define memory access permission. It monitors transactions, including instruction fetches and data accesses from the processor, which can trigger a fault exception when an access violation is detected.

- **Saturation arithmetic** is a version of arithmetic in which all operations such as addition and multiplication are limited to a fixed range between a minimum and maximum value.

Dr. Raghudathesh GP